

توضیحات پروژه درخت تصمیم

ملیکا پهلوانی

402521099

بخش اول داک :

توضیحات بلاک های مختلف کد

بخش دوم داک :

بقیه توضیحات و منابع متفرقه

```

class Node():
    # part1
    def __init__(self, feature=None, children=None, value=None, feature_value=None):
        self.feature = feature
        self.children = children
        # if it is leaf
        self.value = value
        # for part 4
        self.feature_value=feature_value

```

یک کلاس نود برای ساخت درخت و نودهای داخل درخت تصمیم ساختیم

هر نود داخل درخت تصمیم می تونه برگ باشه یا می تونه ویژگی باشه که نمونه ما رو تقسیم میکنه
 اگر نود یک برگ رو مشخص کنه پس باید یک VALUE داشته باشه (اگر درخت تصمیم برای دسته های
 دو حالت باشه این میشه VALUE یعنی مثل پروژه که خروجی ما رضایت یا عدم رضایت بود در غیر این
 صورت میشه CLASSIFICATION)

اگر نود برگ نباشه یعنی بچه هایی داره که به واسطه ی اون شاخه های دیگه درخت شکل می گیرن
 برای همین یک لیست به نام CHILDREN برای نگه داشتن بچه های یک نود داریم و فیچر میشه نام اون
 ویژگی که این نود جدید رو در درخت ایجاد کرده

برای پارت چهارم و پنجم پروژه که باید در درخت جست و جو انجام می دادیم FEATURE_VALUE هم
 به کلاس نود اضافه شد تا بدونیم هر نودی با کدوم ویژگی پدرش از اون جدا شده

```
class DecisionTree():
    # part2
    def __init__(self, mode=None, max_Depth=float("inf"), min_Samples=2,
                  pruning_threshold=None, root=None):
        self.mode = mode
        self.max_Depth = max_Depth
        self.min_Samples = min_Samples
        self.pruning_threshold = pruning_threshold
        self.root = root
```

بخش اول ساخت درخت با تعیین کردن ویژگی های اون شروع میشه

درخت تصمیم باید MODE داشته باشه که مشخص میکنه بهترین ویژگی که قراره درخت طبق اون جلو بره و ساخته بشه با چه الگوریتمی قراره پیدا بشه توی درخت تصمیم پروه ما دو MODE وجود داشت

GING ,GAIN که بعدا به کدش میرسیم

MAX_DEPTH ,MIN_SAMPLES ویژگی هایی هستن که به ما کمک میکنن جلوی بیشپردازش رو بگیریم و نذاریم درخت بیش از حد پیچیده بشه و رشد بکنه

PURNING_THRESHOLD برای هرس و بعد از ساخت درخت تصمیمه

و ROOT هم ریشه ی درخته که اولین تقسیم روی داده ها رو نشون میده

```
def _create_Tree(self, X, Y, depth=0):
    num_Samples = len(Y)
    # if all in one class
    if (len(np.unique(Y))==1) or len(X.columns) == 0:
        return Node(value=self._calculate_Value(Y))

    if depth >= self.max_Depth or num_Samples < self.min_Samples or len(X.columns) == 0:
        return Node(value=self._calculate_Value(Y))

    best_Feature = self._get_best_Feature(X, Y)
    if best_Feature is None or best_Feature.feature is None :
        return Node(value=self._calculate_Value(Y))

    children = []
    fr = best_Feature.feature
    for c in X[fr].unique():
        xi = X[X[fr]==c].drop(columns=[fr])
        yi = Y[X[fr]==c]

        if len(yi)<self.min_Samples:
            child = Node(value=self._calculate_Value(yi), feature_value=c)
            # child = Node(value=self._calculate_Value(yi))
        else:
            child = self._create_Tree(xi,yi,depth+1)
            child.feature_value = c
        children.append(child)
    return Node(feature=best_Feature.feature,children=children,value=self._calculate_Value(Y),feature_value=best_Feature.feature_value)
```

تابع ساخت درخت اصلی ترین و پرچالش ترین بخش کلاس درخت تصمیم بود در بخش اول میاد شرط هایی رو بررسی میکنه که در ازای اون ما باید یک برگ تولید کنیم اگر VALUE همه داده های ما یکی باشه باید یک نود با ارزش همون کلاس ریترن کنیم.

اگر عمق درخت بیشتر از چیزی شده باشه که ما تعیین کردیم و یا تعداد داده های باقی مونده کمتر از حدی باشه که ما گذاشتیم باز هم باید با تقریب ارزش داده های باقی مونده رو به صورت یک برگ برگردونیم

اگر هیچ کدوم از این شرط ها برقرار نبود ادامه می دهیم و بهترین ویژگی رو پیدا می کنیم ممکنه دیگه ویژگی برای تقسیم کردن نمونه شده در اون صورت باز هم با تقریب ارزش داده های باقی مونده رو حساب میکنیم و به صورت یک برگ برگردونیم

با توجه به دسته بندی که بهترین ویژگی داره ما داده ها بخش بندی میکنیم

هر بخش یه بچه برای نود پدرش میشه

اگر توی اون دسته بندی تعداد داده ها کمتر از حدی باشه که ما قرار دادیم برگ رو برمی گردونیم در غیر این صورت ما تابع رو به صورت بازگشتی برای نمونه ها اجرا میکنیم

```
def _information_Gain(self, feature, X, Y):
    def entropy(y):
        en = 0
        classes = np.unique(y)
        for c in classes:
            pc = (np.sum(y == c) / len(y))
            if pc > 0 and pc < 1:
                en += -(np.log2(pc) * pc)
            elif pc == 0:
                en += 0
            elif pc == 1:
                return 0
        return en

    rem = 0
    for value in feature.unique():
        rem += ((len(Y[feature == value]) / len(feature)) * (entropy(Y[feature == value])))
    return entropy(Y) - rem

# part3.2
def _gini_Split(self, feature, X, Y):
    def gini(y):
        gin = 0
        classes = np.unique(y)
        for c in classes:
            pc = np.sum(y == c) / len(y)
            gin += pc ** 2
        return 1 - gin

    ginall = 0
    for value in feature.unique():
        ginall += (len(Y[feature == value]) / len(feature)) * gini(Y[feature == value])
    return ginall
```

توابع محاسباتی که برای پیدا کردن بهترین ویژگی از اون ها استفاده می کنیم

اول چون دیتاست داده شده دو کلاس بود من دو کلاس نوشتم ولی چون تست کیس ها برای چند کلاس هم طراحی شده بودن تغییر دادم

```

# part3.3
def _get_best_Feature(self, X, Y):
    bestf = Node()
    if self.mode == "gain":
        best = -1000
        for feature in X.columns:
            test = self._information_Gain(X[feature], X, Y)
            if test > best:
                best = test
                bestf.feature = feature
        return bestf
    elif self.mode == "gini":
        best = 0.5
        bestf = Node()
        for feature in X.columns:
            test = self._gini_Split(X[feature], X, Y)
            if test < best :
                best = test
                bestf.feature = feature
        return bestf

```

این تابع با توجه به MODE درخت بهترین ویژگی رو پیدا میکنه

```

# if all features done or we create child with no Y or max_depth
def _calculate_Value(self, Y):
    return max(set(Y), key=list(Y).count)

# part4
def fit(self, X_Train, Y_Train):
    self.root = self._create_Tree(X_Train, Y_Train)

```

دو تا تابع کمکی آماده که اولی مقدار برگها رو در شرطهای تابع درخت تصمیم که به ساخت نود برگ منتهی میشه تقریب میزنه و مشخص میکنه

تابع FIT هم که داده های آموزشی رو میگیره و به کمک ویژگی ROOT شروع به ساخت درخت تصمیم میکنه

```

def predict(self, X):
    def _move_Tree(sample, root):
        if root.children == None :
            return root.value
        # if root.value != None :
        #     return root.value
        else:
            for c in root.children :
                if sample[root.feature] == c.feature_value:
                    return _move_Tree(sample,c)
    # for s in X.columns:
    #     _move_Tree(s,self.root)
    preds = []
    for i in range(len(X)):
        sample = X.iloc[i]
        preds.append(_move_Tree(sample,self.root))
    return preds

```

تابع احتمال برای درخت تصمیم

بعد از این که داده ی آموزشی رو به تابع درخت دادیم و درخت تصمیم رو ساختیم داده تستی که می‌خواهیم رو میدیم و براساس درختی که ساختیم VALUE ها رو ارزشگذاری می‌کنیم این تابع برای هر نمونه از ریشه شروع میکنه و هر بار تا عمق درخت برای اون نمونه پیش میره تا ارزش احتمالی رو پیدا کنه

(این عکس مال قبل از تست و ترین هست برای همین یه خطش فرق داره)

```

def calculate_accuracy(tree, X, y):
    predictions = tree.predict(X)
    return np.mean(predictions == y)

def grid_search(X_train, y_train, X_val, y_val):
    best_params = {}
    best_score = 0
    results = []
    max_depths= [6, 8, 10, 12]
    min_samples= [15, 20, 25, 30, 50]
    # max_depths = [8]
    # min_samples = [25]
    modes = ['gain']

    for depth in max_depths:
        for min_sample in min_samples:
            for mode in modes:
                tree = DecisionTree(
                    mode=mode,
                    max_Depth=depth,
                    min_Samples=min_sample
                )

                tree.fit(X_train, y_train)
                accuracy = calculate_accuracy(tree, X_val, y_val)

                print(f"Depth: {depth}, Min Samples: {min_sample}, Mode: {mode} -> Accuracy: {accuracy}")

                results.append({
                    'max_depth': depth,
                    'min_samples': min_sample,
                    'mode': mode,
                    'accuracy': accuracy
                })

                if accuracy > best_score:
                    best_score = accuracy
                    best_params = {
                        'max_depth': depth,
                        'min_samples': min_sample,
                        'mode': mode
                    }

    return best_params, best_score, results

```

بعد از اینکه ساختن بدنه ی اصلی کد تموم شد باید کدهایی نوشته می شد که به بهبود کارایی الگوریتم ختم میشد

یکی از اون روشها روش GRID_SEARCH برای پیدا کردن بهترین هایپرپارامتر هست

می تونیم خودمون به صورت رندوم برای درختمون MAX_DEPTH یا MIN_SAMPLES رو انتخاب کنیم اما هم برای درک بهتر خودمون و هم برای بهتر شدن درصد ها بهتره این کار رو با گرید سرچ انجام بدیم

اعدادی که فکر میکنیم ممکنه مناسب باشن رو داخل لیست قرار می دییم و همه ی ترکیب های ممکن برای اونها رو امتحان می کنیم تا بهترین ترکیب رو پیدا کنیم

و داده های آموزشی رو با همون ترکیب بسازیم و درخت نتیجه بهتری بده

```
# print(f"Number of duplicate rows: {num_duplicates}")
# =====
# HELPER FUNCTIONS FOR PREPROCESSING
# =====
def dropuselesscolumns(df):
    if 'id' in df.columns:
        df = df.drop(columns=['id'])
    return df

def capoutliers(df, numeric_cols):
    for col in numeric_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df[col] = df[col].clip(lower, upper)
    return df

def fillmissingnumeric(df):
    for col in df.select_dtypes(include=['float64', 'int64']).columns:
        df[col] = df[col].fillna(df[col].median())
    return df

def fillmissingcategorical(df):
    for col in df.select_dtypes(include=['object']).columns:
        df[col] = df[col].fillna(df[col].mode()[0])
    return df

def removeduplicates(df):
    return df.drop_duplicates()

def encodecategoricallabels(df, cat_cols):
    for col in cat_cols:
        df[col] = df[col].astype(object)
        df[col] = df[col].fillna('MISSING')
        df[col] = pd.Categorical(df[col]).codes
    return df
```

dropuselesscolumns

تابعی که ستون آیدی را حذف می کند چون هیچ اطلاعات مفیدی برای ساخت درخت نمی دهد

capoutliers

به کمک تابع آماده پانداس چارک اول و سوم را حساب می کند و اینطوری که بازه ایجاد می کند که به کمک آن می توان به داده های پرت مقدار داد

تکنیک cap برای داده های پرت را جایگزین حذف کامل آنها کردم، چون با تابعی که قبلا برای حذف داده های پرت نوشته بودم بیشتر از نصف داده ها از بین می رفتن

fillmissingnumeric

برای داده های ستون های عددی که null هستند از میانه استفاده می کنیم

fillmissingcategorical

برای داده هایی از ستون های آبجکت که null هستند از مد استفاده می کنیم(بیشترین)

removeduplicates

این تابع برای این داده ها بی فایده است چون بین ردیف ها تکراری وجود نداشت

ولی در کل ردیف های تکراری رو حذف می کنه

encodecategoricallabels

این تابع مهم ترین بخشی بود که بعد از تست و ترین اول انجامش دادم

این تابع ستون های string را به اعداد تبدیل می کند

(برای مثال اگر شهر ها بودند هر شهر به یه عدد نگاشت میشه

در روش قبلی به جای این کار من برای هر شهر یه ستون صفر و یک در نظر می گرفتم که اینطوری تعداد ستون ها خیلی زیاد میشد)

```
# =====  
# FUNCTIONS  
# =====  
def smart_age_binning(df):  
    bins = [0, 18, 30, 45, 60, 100]  
    labels = ['Child_Teen', 'Young_Adult', 'Adult', 'Middle_Aged', 'Senior']  
    df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels)  
    return df  
  
def smart_delay_binning(df):  
    dep_bins = [-1, 0, 15, 60, 120, float('inf')]  
    dep_labels = ['No_Delay', 'Short_Delay', 'Medium_Delay', 'Long_Delay', 'Very_Long_Delay']  
    df['Departure_Delay_Level'] = pd.cut(df['Departure Delay in Minutes'],  
                                        bins=dep_bins, labels=dep_labels)  
  
    arr_bins = [-1, 0, 15, 60, 120, float('inf')]  
    arr_labels = ['No_Delay', 'Short_Delay', 'Medium_Delay', 'Long_Delay', 'Very_Long_Delay']  
    df['Arrival_Delay_Level'] = pd.cut(df['Arrival Delay in Minutes'],  
                                      bins=arr_bins, labels=arr_labels)  
    return df  
  
def smart_distance_binning(df):  
    q1 = df['Trip Distance'].quantile(0.25)  
    q2 = df['Trip Distance'].quantile(0.5)  
    q3 = df['Trip Distance'].quantile(0.75)  
    bins = [0, q1, q2, q3, float('inf')]  
    labels = ['Short', 'Medium', 'Long', 'Very_Long']  
    df['Distance_Category'] = pd.cut(df['Trip Distance'], bins=bins, labels=labels)  
    return df  
  
def smart_train_age_binning(df):  
    current_year = 2024  
    df['Train_Age'] = current_year - df['Train Model Year']  
    bins = [0, 10, 20, 30, 50, 100]  
    labels = ['New', 'Young', 'Middle', 'Old', 'Very_Old']  
    df['Train_Age_Group'] = pd.cut(df['Train_Age'], bins=bins, labels=labels)  
    return df
```

این توابع در واقع ستون های عددی رو دسته بندی می کنن تا کار با ستون ها راحت تر بشه و از نویز ها یا همون داده های پرت بیشتر جلوگیری بشه

بعد از اینکه درصد داده های test از 60 بالا نمی اومد این بازه بندی رو اضافه کردم و درصد به 80 رسید

```

# =====
# PIPELINE
# =====
full_Data = full_Data.dropna(subset=['satisfaction'])
full_Data = dropuselesscolumns(full_Data)

numeric_cols = [
    'Departure Delay in Minutes',
    'Arrival Delay in Minutes',
    'Ticket Verification Service',
    'Train Staff Service',
    'Trip Distance'
]

full_Data = capoutliers(full_Data, numeric_cols)
full_Data = fillmissingnumeric(full_Data)
full_Data = fillmissingcategorical(full_Data)
full_Data = removeduplicates(full_Data)

full_Data = smart_age_binning(full_Data)
full_Data = smart_delay_binning(full_Data)
full_Data = smart_distance_binning(full_Data)
full_Data = smart_train_age_binning(full_Data)

categorical_cols = [
    'Passenger Type', 'Type of Trip', 'Ticket Class', 'nation', 'satisfaction',
    'Age_Group', 'Departure_Delay_Level', 'Arrival_Delay_Level',
    'Distance_Category', 'Train_Age_Group'
]
full_Data = encodecategoricalallabels(full_Data, categorical_cols)

# =====
# Save cleaned dataset
# =====
full_Data.to_csv('EuroRail_Survey_cleaned.csv', index=False)
print("\n Cleaned Data Info:")
print(full_Data.info())
print('\nPreprocessing completed successfully!')

```

بعد از اینکه همه ی توابع رو ساختیم به ترتیب مراحل DATA CLEANING رو انجام می دیم

اول اون ردیف هایی که ستون رضایتمندی اونها null هست رو حذف می کنیم چون قطعاً برای ساخت درخت مفید نیستن با اینکار درصد test از 80 به 83 رسید

بعد ستون آیدی رو حذف می کنیم داده های عددی پرت رو مقدار دهی می کنیم و داده های خالی رو پر می کنیم و بعد از اون بازه بندی برای ستون هایی که خودمون مشخص کردیم استفاده می کنیم و بعد همه ی ستون های string رو به ستون های عددی تبدیل می کنیم

داده های جدید رو ذخیره می کنیم

```

# =====
# MODEL TRAINING & TESTING
# =====
# Load cleaned dataset
full_Data = pd.read_csv('EuroRail_Survey_cleaned.csv')
# Split features and target variable
X_Data = full_Data.drop(columns=['satisfaction'])
Y_Data = full_Data['satisfaction']
# Split data into training+validation and testing sets
X_TrainVal, X_Test, Y_TrainVal, Y_Test = train_test_split(
    X_Data, Y_Data, test_size=0.1, random_state=42)
# Further split training data for validation
X_Train, X_Val, Y_Train, Y_Val = train_test_split(
    X_TrainVal, Y_TrainVal, test_size=0.2, random_state=42)
# Grid search to find best hyperparameters
best_params, best_score, results = grid_search(X_Train, Y_Train, X_Val, Y_Val)
print("\nBest Hyperparameters from Grid Search:", best_params)
print("Best Validation Accuracy from Grid Search:", best_score)
# Train final model on full training+validation set with best hyperparameters
final_tree = DecisionTree(
    mode=best_params['mode'],
    max_Depth=best_params['max_depth'],
    min_Samples=best_params['min_samples']
)
# final_tree = DecisionTree(
#     mode='gini',
#     max_Depth=7,
#     min_Samples=20
# )
final_tree.fit(X_TrainVal, Y_TrainVal)
# Make predictions
train_pred = final_tree.predict(X_TrainVal)
test_pred = final_tree.predict(X_Test)
val_pred = final_tree.predict(X_Val)
# Calculate accuracies
train_accuracy = np.mean(train_pred == Y_TrainVal)
test_accuracy = np.mean(test_pred == Y_Test)
val_accuracy = np.mean(val_pred == Y_Val)
print('\nTrain Accuracy:', train_accuracy)
print('Validation Accuracy:', val_accuracy)
print('Test Accuracy:', test_accuracy)

```

در اینجا اول داده ها رو تقسیم بندی می کنیم هم برای گزید سرچ و هم برای داده های آموزشی و داده های تست

بعد از گزید سرچ استفاده می کنیم تا بیشترین عمق درخت ، کمترین نمونه و الگوریتم پیدا کردن بهترین ویژگی رو پیدا کنیم بعد درخت رو برای مدل می سازیم و نتایج رو چاپ می کنیم

```

Depth: 8, Min Samples: 20, Mode: gini -> Accuracy: 0.8093005269009523

Best Hyperparameters from Grid Search: {'max_depth': 8, 'min_samples': 20, 'mode': 'gini'}
Best Validation Accuracy from Grid Search: 0.8093005269009523

Train Accuracy: 0.9222402691571436
Validation Accuracy: 0.9238652293711026
Test Accuracy: 0.8341003132371708

Model training and testing completed!

```

```
def visualize_tree_smart(root, max_children_display=5, filename="decision_tree"):

    dot = Digraph(comment='Decision Tree')
    dot.attr('node', shape='box', style='rounded', fontname='Arial', fontsize='10')
    dot.attr('edge', fontname='Arial', fontsize='8')
    dot.attr(bgcolor="white")

    def add_node(node, parent_id=None, edge_label=""):
        node_id = str(id(node))

        if node.children and len(node.children) > max_children_display:
            label = f"{node.feature}\n({len(node.children)} children)"
            if node.value is not None:
                label += f"\nvalue={node.value}"
            dot.node(node_id, label, fillcolor='orange', style='filled')

            for i, child in enumerate(node.children[:max_children_display]):
                if hasattr(child, 'feature_value') and child.feature_value is not None:
                    child_edge_label = str(child.feature_value)
                else:
                    child_edge_label = ""
                add_node(child, node_id, child_edge_label)

            remaining_id = f"{node_id}_remaining"
            dot.node(remaining_id, f"... {len(node.children) - max_children_display} more",
                    fillcolor='lightgray', style='filled')
            dot.edge(node_id, remaining_id, label="...")

        elif node.children:
            label = f"{node.feature}"
            if node.value is not None:
                label += f"\n(value={node.value})"
            dot.node(node_id, label, fillcolor='lightblue', style='filled')

            for child in node.children:
                if hasattr(child, 'feature_value') and child.feature_value is not None:
                    child_edge_label = str(child.feature_value)
                else:
                    child_edge_label = ""
                add_node(child, node_id, child_edge_label)

        else:
            label = f"Class: {node.value}"
            dot.node(node_id, label, fillcolor='lightgreen', style='filled')

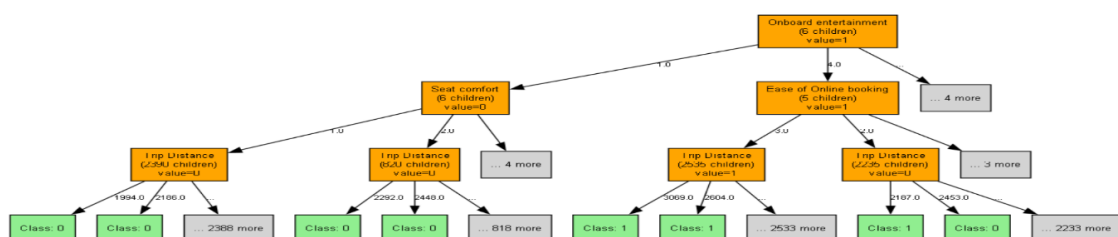
        if parent_id:
            dot.edge(parent_id, node_id, label=edge_label)

    add_node(root)
    dot.render(filename, format='png', cleanup=True)
    print(f"Tree visualization saved as {filename}.png")
    return dot

dot = visualize_tree_smart(final_tree.root, max_children_display=4, filename="final_tree_visual")
```

بخش نمایش درخت به صورت گرافیکی

چون درخت نسبتاً بزرگ بود رسمش به صورت گرافیکی خیلی سخت بود برای همین کد اینجوریه که خیلی ساده سازی میکنه ولی امکان مدیریتش هم هست



CSS
JAVASCRIPT
SQL
PYTHON
JAVA
PHP
HOW TO
W3.CSS
C
C++
C#
BOOTSTRAP
REACT
MYSQL
JQUERY
EXCEL
XML

NumPy Tutorial
NumPy HOME
NumPy Intro
NumPy Getting Started
NumPy Creating Arrays
NumPy Array Indexing
NumPy Array Slicing
NumPy Data Types
NumPy Copy vs View
NumPy Array Shape
NumPy Array Reshape
NumPy Array Iterating
NumPy Array Join
NumPy Array Split
NumPy Array Search
NumPy Array Sort
NumPy Array Filter
NumPy Random
Random Intro
Data Distribution
Random Permutation

NumPy Tutorial

[< Home](#)
[Next >](#)

Learn NumPy

NumPy is a Python library.

NumPy is used for working with arrays.

NumPy is short for "Numerical Python".

Learning by Reading

We have created 43 tutorial pages for you to learn more about NumPy.

Starting with a basic introduction and ends up with creating and plotting random data sets, and working with NumPy functions:

پیدا کردن فرمول ها و توضیحات و بعضی از اشکالاتی از این کتابخانه داشتم

S
JAVASCRIPT
SQL
PYTHON
JAVA
PHP
HOW TO
W3.CSS
C
C++
C#
BOOTSTRAP
REACT
MYSQL
JQUERY
EXCEL
XML
DJANGO
NUMPY

Pandas Tutorial
Pandas HOME
Pandas Intro
Pandas Getting Started
Pandas Series
Pandas DataFrames
Pandas Read CSV
Pandas Read JSON
Pandas Analyzing Data
Cleaning Data
Cleaning Data
Cleaning Empty Cells
Cleaning Wrong Format
Cleaning Wrong Data
Removing Duplicates
Correlations
Pandas Correlations
Plotting

Pandas Tutorial

[< Home](#)
[Next >](#)

Learn Pandas

Pandas is a Python library.

Pandas is used to analyze data.

Learning by Reading

We have created 14 tutorial pages for you to learn more about Pandas.

Starting with a basic introduction and ends up with cleaning and plotting data:

Videos

Decision and Classification Trees, Clearly Explained!!!

YouTube · StatQuest with Josh Starmer · Ordibehesht 6, 1400 AP

Learn how to build and use **decision trees** for classification. **Decision trees** make statements and classify things based on those statements.

How To Use A Decision Tree To Predict Classes In Power BI ...

YouTube · Enterprise DNA · Estand 22, 1401 AP

Visualize **decision tree** classifier in Power BI to predict flower species (Sentosa, Versicolor, Virginica) based on petal and sepal dimensions.

Decision Trees - Visually Explained

YouTube · Visually Explained · Farvardin 10, 1404 AP

Decision trees are simple yes/no question-based machine learning models that can be used for data classification.

استفاده از chatgpt و بقیه llm ها در حدی که بعضی وقت ها منطق یه کاری رو میپرسیدم یا اگر تابع ها به خاطر سینتکس ها ارور می خوردن کمک می گرفتم

توی بخش رسم درخت هم چون دیر شده بود بخش بخش پرسیدم که دقیقا برای نصبش و اینها باید چی کار کنم

مشکلات over fit :

اولین مشکل که درصد داده های test رو تا 60 آورده بود پایین مسئله پیش پردازش بود که دقیق انجام نمی شد یا تعداد زیادی از داد ها رو از بین می بردم یا داده های عددی خوب بازه بندی نمی شدن

دومین مشکل برای مقداردهی های بد برای حداقل نمونه های باقی مانده بود که تعدادشون رو کم می دادم و هم زمان زیادی طول میکشید و هم دقت رو می آورد پایین

وقتی مدل ترتیب دادن به دیتاست رو عوض کردم و اعداد بهتری رو برای گرید سرچ امتحان کردم درصد test به 83 رسید

اهدافی که بهش نرسیدم:

میخواستم چک کنم که هر ویژگی فقط یک بار استفاده بشه و در عمق دیگری نباشه ولی با اینکار نمی دونم چرا تستهای پارت چهارم پاس نمیشد

الگوریتم هرسم نتونست به بهتر شدن درصد تستم کمک بکنه